34.2 媒体管道

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~58k tokens，输出 ~8k tokens（本节）

聊天机器人不只处理文字——用户会发送图片、音频、PDF 文档，Agent 也可能回复带有媒体附件的消息。OpenClaw 的媒体管道（ `src/media/` ）负责整个媒体生命周期：从远程获取、MIME 类型检测、本地存储、图像缩放与格式转换，到通过 HTTP 服务器对外提供临时下载链接。本节将逐一拆解这条管道上的每个环节。

34.2.1 媒体获取（ `src/media/fetch.ts` ）

远程媒体拉取

当 Agent 在回复中输出 `MEDIA: https://example.com/photo.jpg` 这样的令牌，Gateway 需要将该 URL 指向的资源下载到本地。 `fetchRemoteMedia` 是这条路径的入口函数：

```typescript
// src/media/fetch.ts（简化）
export type FetchMediaResult = {
  buffer: Buffer;        // 文件二进制内容
  contentType?: string;  // 检测后的 MIME 类型
  fileName?: string;     // 推断出的文件名
};

export async function fetchRemoteMedia(options: FetchMediaOptions): Promise<FetchMediaR
  // 1. SSRF 防护：通过 fetchWithSsrFGuard 发起请求
  const { response, finalUrl, release } = await fetchWithSsrFGuard({
    url: options.url,
    fetchImpl: options.fetchImpl,
    maxRedirects: options.maxRedirects,
    policy: options.ssrfPolicy,
    lookupFn: options.lookupFn,
  });

  // 2. HTTP 状态检查（非 2xx 抛出 MediaFetchError）
  if (!response.ok) {
    throw new MediaFetchError("http_error", `HTTP ${response.status}...`);
  }

  // 3. 预检 Content-Length，超限则提前拒绝
  if (maxBytes && contentLength > maxBytes) {
    throw new MediaFetchError("max_bytes", `content length exceeds maxBytes`);
  }

  // 4. 流式读取 + 实时字节限制
  const buffer = maxBytes
    ? await readResponseWithLimit(response, maxBytes)
    : Buffer.from(await response.arrayBuffer());

  // 5. 文件名推断：Content-Disposition > URL 路径 > filePathHint
  const fileName = headerFileName || fileNameFromUrl || filePathHint;

  // 6. MIME 检测 + 扩展名补全
  const contentType = await detectMime({ buffer, headerMime, filePath });
  return { buffer, contentType, fileName };
}
```

SSRF 防护

> **衍生解释 — SSRF（Server-Side Request Forgery）**
>
> SSRF 是一种安全漏洞：攻击者让服务器代为请求内部网络资源（如 `http://169.254.169.254/` 获取云元数据）。OpenClaw 通过 `fetchWithSsrFGuard` 进行 DNS 预解析，拒绝指向私有 IP 地址段（10.x、172.16-31.x、192.168.x）的请求，并限制重定向次数，防止通过 302 跳转绕过检查。

流式字节限制

`readResponseWithLimit` 使用 `ReadableStream` 的 `getReader()` API 逐块读取响应体。每读入一块数据，累加字节计数器。一旦超过 `maxBytes` 阈值，立即调用 `reader.cancel()` 终止流传输并抛出 `max_bytes` 错误——这避免了先将整个大文件下载到内存再检查大小的浪费：

```
// src/media/fetch.ts
async function readResponseWithLimit(res: Response, maxBytes: number): Promise<Buffer>
  const reader = res.body!.getReader();
  const chunks: Uint8Array[] = [];
  let total = 0;
  while (true) {
    const { done, value } = await reader.read();
    if (done) break;
    total += value.length;
    if (total > maxBytes) {
      await reader.cancel();
      throw new MediaFetchError("max_bytes", `payload exceeds maxBytes ${maxBytes}`);
    }
    chunks.push(value);
  }
  return Buffer.concat(chunks.map(c => Buffer.from(c)), total);
}
```

文件名推断策略

文件名的确定遵循三级优先级：

优先级	来源	示例
1（最高）	`Content-Disposition` 响应头	`filename*=UTF-8''%E5%9B%BE%E7%89%87.png`

优先级	来源	示例
2	URL 路径的 `basename`	`https://cdn.example.com/img/photo.jpg` → `photo.jpg`
3（最低）	调用方传入的 `filePathHint`	用户消息中的原始路径

`Content-Disposition` 头支持 RFC 5987 的 `filename*` 扩展编码格式（`charset'language'encoded-value`），`parseContentDispositionFileName` 会先尝试解码 `filename*` 参数，再退回到普通 `filename`。如果推断出的文件名缺少扩展名，还会根据检测到的 MIME 类型自动补全（如 `image/jpeg` → `.jpg`）。

34.2.2 媒体解析（`src/media/parse.ts`）

MEDIA 令牌协议

Agent 在输出文本中嵌入形如 `MEDIA: <url>` 的令牌来指示 Gateway 附带媒体。`splitMediaFromOutput` 负责从 Agent 的原始输出中提取这些令牌，同时保留其余文本内容：

```
// src/media/parse.ts
export const MEDIA_TOKEN_RE = /\bMEDIA:\s*`?([^\n]+)`?/gi;

export function splitMediaFromOutput(raw: string): {
  text: string;           // 去除令牌后的纯文本
  mediaUrls?: string[];   // 提取到的媒体 URL 列表
  mediaUrl?: string;      // 第一个 URL（向后兼容）
  audioAsVoice?: boolean;
}
```

解析流程

解析过程需要处理多种边界情况：

1. 围栏代码块保护：如果 `MEDIA:` 出现在 Markdown 的 ```` ``` ```` 围栏代码块内部，它是代码示例而非真实的媒体令牌。解析器先调用 `parseFenceSpans` 计算所有围栏区间，逐行扫描时检查当前行的字符偏移是否落在某个围栏区间内——如果是，则跳过提取。

2. URL 校验与清洗：提取到的候选值经过 `cleanCandidate（去除包裹的引号、括号）和 `isValidMedia` 校验。合法的媒体源只有两种形式：

- HTTP(S) URL： `https://example.com/image.png`
- 安全的相对路径：必须以 `./` 开头，且不含 `..` （禁止目录遍历）

3. 含空格路径的回退策略：当令牌值看起来像本地路径（以 `/` 、 `./` 、 `~/` 、 `file://` 开头）且包含空格时，解析器会尝试将整个值（而非按空格分割后的片段）作为单个路径来校验。

4. 语音标签检测：解析完毕后，还会调用 `parseAudioTag` 检查文本中是否包含 `[[audio_as_voice]]` 指令标签。这个标签告诉 Gateway 将音频以"语音消息气泡"（而非普通文件附件）的形式发送到聊天平台。

清洗后的文本

提取完所有 `MEDIA:` 行后，剩余文本会经过多轮清洗：

```
原始行 → 去除 MEDIA 令牌 → 合并多余空白 → 去除连续空行 → 去除 [[audio_as_voice]] 标签
```

最终返回的 `text` 字段是干净的、可以直接发送给用户的纯文本/Markdown 内容。

34.2.3 媒体存储（ `src/media/store.ts` ）

存储目录

媒体文件保存在 OpenClaw 配置目录下的 `media/` 子目录中（通常是 `~/.config/openclaw/media/` ），由 `ensureMediaDir` 按需创建，权限设为 `0o700` （仅当前用户可访问）。

存储方式

`store.ts` 提供两个存储入口：

函数	输入	用途
`saveMediaSource(source)`	URL 或本地路径	Agent 输出的 `MEDIA:` 令牌指向的资源
`saveMediaBuffer(buffer)`	内存中的 Buffer	平台接收到的入站媒体（用户发送的图片等）

`saveMediaSource` 的双路径处理：

```
// src/media/store.ts（简化）
export async function saveMediaSource(source: string): Promise<SavedMedia> {
  const baseId = crypto.randomUUID();

  if (looksLikeUrl(source)) {
    // URL 路径：流式下载到临时文件 → MIME 检测 → 重命名为最终文件
    const tempDest = path.join(dir, `${baseId}.tmp`);
    const { headerMime, sniffBuffer, size } = await downloadToFile(source, tempDest, he
    const mime = await detectMime({ buffer: sniffBuffer, headerMime, filePath: source }
    const ext = extensionForMime(mime) ?? path.extname(new URL(source).pathname);
    await fs.rename(tempDest, path.join(dir, `${baseId}${ext}`));
    return { id, path: finalDest, size, contentType: mime };
  }

  // 本地路径：直接读取文件到内存 → MIME 检测 → 写入存储目录
  const buffer = await fs.readFile(source);
  const mime = await detectMime({ buffer, filePath: source });
  await fs.writeFile(dest, buffer, { mode: 0o600 });
  return { id, path: dest, size: stat.size, contentType: mime };
}
```

`saveMediaBuffer` 的原始文件名嵌入：接收入站媒体时，原始文件名通过 `{sanitized}---{uuid}.{ext}` 格式嵌入到存储文件名中。 `sanitizeFilename` 会过滤掉跨平台不安全字符（只保留字母、数字、 `.` 、 `-` 、 `_` ），截断到 60 字符。反向提取时， `extractOriginalFilename` 通过正则匹配 `---{uuid}` 模式还原原始名称。

TTL 清理机制

所有媒体文件都是临时性的，默认 TTL 为 2 分钟。 `cleanOldMedia` 在每次保存新文件时触发，遍历存储目录中的所有文件，删除修改时间超过 TTL 的过期文件：

```
export async function cleanOldMedia(ttlMs = 2 * 60 * 1000) {
  const entries = await fs.readdir(mediaDir);
  await Promise.all(entries.map(async (file) => {
    const stat = await fs.stat(full);
    if (Date.now() - stat.mtimeMs > ttlMs) {
      await fs.rm(full).catch(() => {});  // 静默删除
    }
  }));
}
```

流式下载到磁盘

`downloadToFile` 使用 Node.js 原生的 `http.request` / `https.request` （而非 `fetch` ）进行下载，原因是需要精细控制流传输过程：

1. **SSRF 防护**：通过 `resolvePinnedHostname` 对目标主机名进行 DNS 预解析并锁定 IP（pinning），将解析结果传递给底层请求的 `lookup` 参数
2. **手动重定向跟踪**：最多 5 次重定向，每次解析 `Location` 头并递归调用
3. **流式写入**：使用 `pipeline(res, createWriteStream(dest))` 将响应体直接流向磁盘
4. **MIME 嗅探缓存**：同时缓存前 16KB 数据到内存（ `sniffChunks` ），供后续 MIME 检测使用
5. **大小限制**：累计字节超过 5MB（ `MEDIA_MAX_BYTES` ）时调用 `req.destroy()` 终止连接

34.2.4 图像处理：双后端架构（ `src/media/image-ops.ts` ）

Sharp 与 sips 双后端

OpenClaw 的图像处理支持两种后端，根据运行环境自动选择：

后端	条件	技术
Sharp	默认后端（Node.js 环境）	基于 libvips 的高性能图像库
sips	Bun 运行时 + macOS	macOS 内置的 Scriptable Image Processing System

> **衍生解释 — Sharp**
>
> Sharp 是 Node.js 生态中最流行的图像处理库，底层调用 C 库 libvips。它支持 JPEG、PNG、WebP、HEIC 等格式的读取、缩放、旋转、格式转换，性能远超纯 JavaScript 方案（如 Jimp）。但 Sharp 依赖原生二进制模块（ `.node` 文件），在某些运行时（如 Bun）中可能存在兼容性问题。

> **衍生解释 — sips**
>
> sips（Scriptable Image Processing System）是 macOS 自带的命令行图像处理工具，位于 `/usr/bin/sips` 。它通过 Core Image 框架支持 JPEG、PNG、HEIC 等格式的查询与转换。OpenClaw 在 Bun + macOS 组合下优先使用 sips 作为 Sharp 的替代方案，避免原生模块兼容性问题。

后端选择逻辑由 `prefersSips` 函数控制：

```
function prefersSips(): boolean {
  return (
    process.env.OPENCLAW_IMAGE_BACKEND === "sips" ||        // 环境变量强制指定
    (process.env.OPENCLAW_IMAGE_BACKEND !== "sharp" &&      // 未强制 Sharp
     isBun() && process.platform === "darwin")              // Bun + macOS
  );
}
```

核心操作

1. 获取图像元信息

```
export async function getImageMetadata(buffer: Buffer): Promise<ImageMetadata | null> {
  if (prefersSips()) {
    // sips -g pixelWidth -g pixelHeight input.img → 解析标准输出
    return await sipsMetadataFromBuffer(buffer);
  }
  const sharp = await loadSharp();
  const meta = await sharp(buffer).metadata();
  return { width: meta.width, height: meta.height };
}
```

sips 后端需要先将 Buffer 写入临时文件（通过 `withTempDir` 辅助函数创建临时目录，函数返回后自动清理），再调用命令行工具解析输出。

2. EXIF 方向矫正

JPEG 文件的 EXIF 元数据中包含拍摄方向信息（orientation，值 1-8），手机竖拍的照片在原始像素上可能是横向的。OpenClaw 实现了一套纯 JavaScript 的 EXIF 方向读取器 `readJpegExifOrientation`，直接解析 JPEG 二进制格式：

```
JPEG 文件结构:
[FFD8] SOI标记
[FFE1] APP1标记 → [Exif\0\0] 头 → TIFF 头 (II/MM字节序)
  → IFD0 (Image File Directory)
    → Tag 0x0112 = Orientation
      → 值 1-8 对应不同的旋转/翻转组合
```

方向值	变换	sips 参数
1	无变换	（跳过）
2	水平翻转	`-f horizontal`
3	旋转 180°	`-r 180`
6	顺时针旋转 90°	`-r 90`
8	顺时针旋转 270°	`-r 270`

Sharp 后端更简单：调用 `.rotate()` （不带参数）即可根据 EXIF 自动旋转。

3. 图像缩放

`resizeToJpeg` 在缩放前先矫正 EXIF 方向，避免缩放后图像方向错误：

```
export async function resizeToJpeg(params: {
  buffer: Buffer;
  maxSide: number;        // 最大边长（像素）
  quality: number;        // JPEG 质量（1-100）
  withoutEnlargement?: boolean;
}): Promise<Buffer> {
  if (prefersSips()) {
    const normalized = await normalizeExifOrientationSips(buffer);
    return await sipsResizeToJpeg({ buffer: normalized, maxSide, quality });
  }
  // Sharp: .rotate() → .resize({ fit: "inside" }) → .jpeg({ mozjpeg: true })
  return await sharp(buffer).rotate().resize({ width: maxSide, height: maxSide,
    fit: "inside", withoutEnlargement: true }).jpeg({ quality, mozjpeg: true }).toBuffe
}
```

Sharp 使用 `fit: "inside"` 策略——将图像等比缩放到 `maxSide × maxSide` 的包围盒内， `mozjpeg: true` 启用 Mozilla JPEG 编码器以获得更好的压缩比。

4. PNG 优化

对于带透明通道（alpha）的图像，OpenClaw 使用 PNG 格式而非 JPEG。 `optimizeImageToPng` 在一个尺寸-压缩级别的二维搜索网格上寻找满足大小限制的最优组合：

```
尺寸维度:   [2048, 1536, 1280, 1024, 800] 像素
压缩维度:   [6, 7, 8, 9] 级别（0=最快, 9=最小）
```

从最大尺寸开始尝试，找到第一个输出大小 ≤ `maxBytes` 的组合即返回。如果所有组合都超限，返回尝试过程中体积最小的那个结果。

5. HEIC 转换

```
export async function convertHeicToJpeg(buffer: Buffer): Promise<Buffer> {
  if (prefersSips()) {
    // sips -s format jpeg input.heic --out output.jpg
    return await sipsConvertToJpeg(buffer);
  }
  return await sharp(buffer).jpeg({ quality: 90, mozjpeg: true }).toBuffer();
}
```

34.2.5 音频处理（ `src/media/audio.ts` ）

音频模块相对轻量，主要负责判断音频文件是否兼容聊天平台的"语音消息"格式：

```
// src/media/audio.ts
const VOICE_AUDIO_EXTENSIONS = new Set([".oga", ".ogg", ".opus"]);

export function isVoiceCompatibleAudio(opts: {
  contentType?: string | null;
  fileName?: string | null;
}): boolean {
  // 判断条件1：MIME 类型包含 ogg 或 opus
  const mime = opts.contentType?.toLowerCase();
  if (mime && (mime.includes("ogg") || mime.includes("opus"))) return true;

  // 判断条件2：文件扩展名为 .oga / .ogg / .opus
  const ext = getFileExtension(opts.fileName);
  return VOICE_AUDIO_EXTENSIONS.has(ext);
}
```

> **衍生解释 — OGG/Opus 格式**
>
> OGG 是一种开源的多媒体容器格式，Opus 是其中使用最广泛的音频编解码器。Telegram 等聊天平台的语音消息使用 OGG Opus 格式（ `.ogg` 或 `.oga` 扩展名）。相比 MP3，Opus 在低码率下（如 16kbps）依然保持良好的语音质量，非常适合实时通信场景。

配合 `parse.ts` 中的 `[[audio_as_voice]]` 标签，整个音频语音流程为：

```
Agent 输出: "MEDIA: ./voice.ogg\n[[audio_as_voice]]"
  ↓ splitMediaFromOutput 解析
mediaUrl = "./voice.ogg", audioAsVoice = true
  ↓ isVoiceCompatibleAudio 检查格式
true (.ogg 属于语音兼容格式)
  ↓ Gateway 发送为语音气泡（而非文件附件）
```

34.2.6 MIME 类型检测（ `src/media/mime.ts` ）

三源检测策略

MIME 类型的准确检测是整个媒体管道的基石。 `detectMime` 综合三种信息源进行判断：

```
export async function detectMime(opts: {
  buffer?: Buffer;          // 文件二进制内容（用于魔数嗅探）
  headerMime?: string;      // HTTP Content-Type 头
  filePath?: string;        // 文件路径（用于扩展名推断）
}): Promise<string | undefined>
```

> **衍生解释 — 魔数（Magic Number）**
>
> 大多数文件格式在开头几个字节有固定的标识符，称为"魔数"。例如 JPEG 文件以 `FF D8` 开头，PNG 以 `89 50 4E 47` 开头。通过检查这些字节可以准确判断文件类型，不依赖文件扩展名。OpenClaw 使用 `file-type` 库实现魔数检测。

检测优先级：

步骤	来源	说明
1	魔数嗅探（ `fileTypeFromBuffer` ）	最可靠，但无法识别纯文本格式
2	扩展名映射（ `MIME_BY_EXT` ）	覆盖 35+ 常见格式
3	HTTP Content-Type 头	可能不准确，优先级最低

有一个重要的特殊处理：当魔数嚇探返回“通用容器类型”（ `application/octet-stream` 或 `application/zip` ）时，扩展名映射会覆盖嚇探结果。这解决了一个常见问题—— `.xlsx` （Excel 文件）的二进制格式本质上是 ZIP 压缩包，魔数检测会将其识别为 `application/zip` ，而正确的 MIME 应该是 `application/vnd.openxmlformats-officedocument.spreadsheetml.sheet` 。

类型分类与大小限制

`constants.ts` 定义了媒体的四种分类及其大小上限：

分类	MIME 前缀	最大字节
`image`	`image/*`	6 MB
`audio`	`audio/*`	16 MB
`video`	`video/*`	16 MB
`document`	`application/*` （含 PDF）	100 MB

`mediaKindFromMime` 根据 MIME 类型将文件归入以上四类之一， `maxBytesForKind` 返回对应的大小上限。这些限制在媒体获取和存储的各个环节中被一致地执行。

34.2.7 媒体服务器（ `src/media/server.ts` 与 `src/media/host.ts` ）

Express 媒体路由

存储到磁盘的媒体文件需要通过 HTTP 对外提供下载。 `attachMediaRoutes` 在 Express 应用上挂载 `GET /media/:id` 路由：

```
// src/media/server.ts（简化）
export function attachMediaRoutes(app: Express, ttlMs = 2 * 60 * 1000) {
  app.get("/media/:id", async (req, res) => {
    // 1. 校验 ID 格式（仅允许字母数字和 ._-，最长 200 字符）
    if (!isValidMediaId(req.params.id)) return res.status(400).send("invalid path");

    // 2. 安全打开文件（openFileWithinRoot 防止目录遍历）
    const { handle, realPath, stat } = await openFileWithinRoot({
      rootDir: mediaDir, relativePath: id
    });

    // 3. TTL 检查（过期则删除并返回 410 Gone）
    if (Date.now() - stat.mtimeMs > ttlMs) {
      await fs.rm(realPath);
      return res.status(410).send("expired");
    }

    // 4. 返回文件内容（自动检测 MIME 设置 Content-Type）
    const data = await handle.readFile();
    res.type(await detectMime({ buffer: data, filePath: realPath }));
    res.send(data);

    // 5. 单次使用清理：响应完成后 50ms 删除文件
    res.on("finish", () => setTimeout(() => fs.rm(realPath), 50));
  });

  // 定时清理任务
  setInterval(() => cleanOldMedia(ttlMs), ttlMs).unref();
}
```

关键的安全措施包括：

- **ID 格式白名单**：正则 `/^[\p{L}\p{N}._-]+$/u` 确保不含路径分隔符
- **根目录锁定**：`openFileWithinRoot` 解析符号链接后验证真实路径仍在 `mediaDir` 内部
- **410 Gone 响应**：过期文件返回 HTTP 410 而非 404，符合 REST 语义

媒体托管编排

`host.ts` 的 `ensureMediaHosted` 函数编排整个托管流程：

```
saveMediaSource(source)    → 保存文件到磁盘
getTailnetHostname()       → 获取 Tailscale 主机名
isPortFree(42873)          → 检查默认端口是否空闲
  ├─ 端口已占用 → 假设 webhook 服务器已运行，直接返回 URL
  └─ 端口空闲 + startServer=true → 启动临时媒体服务器
最终返回: https://{tailnet-hostname}/media/{id}
```

> **衍生解释 — Tailscale**
>
> Tailscale 是一个基于 WireGuard 的零配置 VPN 网络（也称 Tailnet）。每个加入 Tailnet 的设备会获得一个唯一的主机名（如 `my-server.tailnet-xxxx.ts.net` ）和 HTTPS 证书。OpenClaw 利用 Tailscale 的 Funnel 功能，让本地运行的媒体服务器可以被互联网上的聊天平台（如 Telegram Webhook）访问到。

34.2.8 输入文件处理（ `src/media/input-files.ts` ）

入站文件提取

除了 Agent 输出的出站媒体，OpenClaw 还需要处理用户发送的入站文件。 `input-files.ts` 定义了完整的输入文件处理框架：

```
export type InputFileSource = {
  type: "base64" | "url";   // base64 内联数据 或 URL 引用
  data?: string;             // base64 编码的文件内容
  url?: string;              // 远程文件 URL
  mediaType?: string;        // 声明的 MIME 类型
  filename?: string;
};
```

PDF 提取的双模式策略：

当输入文件为 PDF 时，提取器采用"文本优先，图像兜底"的策略：

1. 使用 `pdfjs-dist` 逐页提取文本内容
2. 如果提取到的文本超过阈值（默认 200 字符），认为 PDF 主要是文本型的，直接返回文本
3. 如果文本过少（扫描件、纯图片 PDF），则加载 `@napi-rs/canvas` 将每页渲染为 PNG 图像

这两个依赖（ `pdfjs-dist` 和 `@napi-rs/canvas` ）都通过懒加载引入——只有真正处理 PDF 时才会 `import()` ，避免非 PDF 场景的启动开销。

安全限制：

参数	默认值	说明
`maxBytes`	5 MB（文件）/ 10 MB（图片）	文件大小上限
`maxChars`	200,000	提取文本的字符上限
`maxPages`	4	PDF 最大处理页数
`maxPixels`	4,000,000	PDF 页面渲染的最大像素数
`maxRedirects`	3	URL 来源的最大重定向次数
`timeoutMs`	10,000	URL 获取超时

本节小结

1. **媒体获取**通过 SSRF 防护的 `fetch` 拉取远程资源，支持流式字节限制和三级文件名推断。
2. **媒体解析**从 Agent 输出中提取 `MEDIA:` 令牌，处理围栏代码块、含空格路径、语音标签等边界情况。
3. **媒体存储**使用 UUID + 原始文件名的命名策略，所有文件默认 2 分钟 TTL，自动清理过期文件。
4. **图像处理**采用 Sharp / sips 双后端架构，覆盖元信息查询、EXIF 方向矫正、JPEG/PNG 缩放、HEIC 转换等操作。
5. **MIME 检测**综合魔数嗅探、扩展名映射、HTTP 头三种来源，通过优先级规则解决 ZIP 容器误判等常见问题。
6. **媒体服务器**提供临时 HTTP 端点，集成 Tailscale Funnel 对外暴露，文件在被下载一次后自动删除。
7. **输入文件处理**支持 base64 / URL 两种来源，PDF 提取采用文本优先、图像兜底的双模式策略。